

WORLDEDGE-FLOW: ADAPTIVE WORLD-MODEL PLANNING FOR DISTRIBUTED EDGE LLM SERVING

Yuhao Zheng^{1,*} Ting You^{2,*}

¹Imperial College London
y.zheng26@imperial.ac.uk

²University of Pennsylvania
tingyou@engineering.upenn.edu

ABSTRACT

World-model planning anticipates the consequences of scheduling decisions, but fixed-horizon rollouts can waste computation on unpromising action sequences. We propose WorldEdge-Flow (WEF), an adaptive planner for distributed edge LLM serving that concentrates prediction effort on promising candidates. WEF screens candidates with short rollouts, selectively deepens a retained subset, and reuses intermediate predictions to avoid redundant computation. Early stopping is guided by action-value separation, prediction uncertainty, and marginal planning gain. On DMControl tasks, WEF achieves the highest mean return among TD-MPC2, SAC, DreamerV3. We then apply WEF to joint request routing and replica scaling under a resource-penalized service-level-objective (SLO) goodput objective. WEF attains 88.93% and 80.45% mean SLO compliance on BurstGPT and WildChat datasets. The relative gains in mean SLO attainment over TD-MPC2, PPO, and SAC are 1.61%, 51.97%, and 8.85% on BurstGPT, and 6.78%, 206.59%, and 24.63% on WildChat, respectively. Frozen-planner ablations further show 61.4% fewer candidate transitions than fixed-horizon planning at $H = 7$, while maintaining higher mean SLO attainment.

Index Terms— World models, adaptive planning, edge LLM serving, reinforcement learning

1. INTRODUCTION

Large language models (LLMs) support interactive applications that require timely responses and sustained generation speed. Deploying these services at distributed edge sites brings computation closer to users, but requires coordinating limited GPU resources across sites [1, 2]. An orchestrator must decide where to serve each arriving request and when to activate or release serving replicas. The goal is to increase service level objective (SLO) goodput, while avoiding unnecessary resource occupancy [3].

LLM workloads make these decisions difficult to coordinate. Requests differ in prompt and response length, and arrivals can be bursty [4, 5]. A short queue does not necessarily imply little remaining work. Moreover, a replica may retain reusable key-value (KV) states from earlier prompts, reducing computation for subsequent requests with a common prefix [6, 7]. Routing to that replica can save computation, but may also increase waiting time if it is busy. Replica scaling introduces another delayed effect: a newly activated replica cannot serve requests until startup finishes, while keeping idle replicas active consumes resources.

Existing systems optimize memory management [6], prefix-aware scheduling [7], and model startup [8]. WEF instead targets the decision layer: predicting how joint routing and scaling affect future

queues, cache reuse, and resource occupancy. Its focus is not a new serving-engine optimization, but selective lookahead that balances scheduling quality against planning computation. These mechanisms complement, rather than replace, the underlying serving systems.

Reinforcement learning offers a way to learn such decisions from interaction [9, 10]. Model-free methods such as PPO [11] and SAC [12] learn policies that directly map observations to actions. Model-based methods instead learn a world model, which predicts how the system evolves under an action, and use it to evaluate possible future action sequences [13, 14, 15, 16]. However, this additional lookahead requires computation for every scheduling decision. A fixed planning horizon extends all candidates to the same depth, even when a short prediction already identifies some as unpromising. The challenge is to retain useful lookahead without spending the same prediction effort on every candidate.

In this paper, we propose WorldEdge-Flow, an adaptive world-model planner for distributed edge LLM serving. Unlike fixed-horizon planning, WEF adapts both which candidates receive further prediction and how far they are expanded. It screens candidates with short rollouts, then selectively deepens a retained subset while reusing intermediate predictions. Early stopping targets the current action decision through action-value separation, critic disagreement, and marginal planning gain. This couples candidate screening with decision-aware stopping, drawing on adaptive-horizon planning [17, 18] and early-exit computation [19, 20]. The distinction is selective prediction across candidates and depths, not merely a shorter horizon.

We first evaluate the planner on six DMControl tasks, then study routing and scaling using BurstGPT [4] and WildChat [5] in a hardware-calibrated simulator. We measure SLO attainment, resource occupancy, and planning computation, with ablations of screening and stopping. WEF achieves higher mean utility than the compared learning baselines on both serving workloads. Our main contributions are:

- We develop an adaptive world-model planner that combines shallow candidate screening, prediction reuse, and early stopping. It concentrates deeper lookahead on promising action sequences instead of uniformly expanding all candidates.
- We formulate joint request routing and replica scaling with queue load, KV-prefix affinity, and replica readiness. A resource-penalized SLO-goodput objective balances timely service against sustained replica occupancy.
- We evaluate WEF on six DMControl tasks and BurstGPT and WildChat datasets, examining control performance, SLO attainment, and resource efficiency. Frozen-planner ablations isolate the contributions of screening and early stopping to planning efficiency.

*Equal contribution.

2. SYSTEM MODEL AND PROBLEM FORMULATION

2.1. Distributed edge LLM serving system

We consider a central gateway and $E = 8$ edge sites. The gateway receives requests and hosts an orchestrator that selects their destinations and controls replica activation. Each edge site provides one GPU and supports two replicas of the same LLM: a resident replica A and an optional replica B. A replica is a containerized serving instance with model weights, a request queue, and local KV state. Replicas at the same site share GPU resources.

Request i arrives at time t_i with prompt length L_i^{in} and realized output length L_i^{out} , which is hidden from the policy. The orchestrator selects $a_i = (e_i, b_i, \delta_i)$: a site, a ready replica $b_i \in \{A, B\}$, and a command $\delta_i \in \{-1, 0, +1\}$ to deactivate, retain, or activate B at that site. Lifecycle feasibility masks the 48 nominal actions. The selected replica queues the request until admission is possible; B admits work only after startup, and deactivation drains existing work.

2.2. Load and KV-cache state

The orchestrator observes prompt length and each replica's queued/active requests, memory occupancy, lifecycle state, and prefix affinity. KV occupancy limits admission, whereas reusable KV state can reduce prefill computation [21, 7]. Let $\ell_{i,e,b}$ be the number of prompt tokens covered by a contiguous cached prefix at replica (e, b) . Its affinity is $\rho_{i,e,b} = \ell_{i,e,b} / \max(1, L_i^{\text{in}})$. WildChat prefixes are identified by cumulative block hashes, with pin-safe LRU eviction and measured cold/warm prefill savings. BurstGPT prefix reuse is disabled because text identity is unavailable.

2.3. Problem formulation

Let D_i^{first} be time to first token (TTFT), D_i^{token} the time per output token (TPOT), and c_i indicate compliance with the supported context limit. Let τ_f and τ_p denote the SLO thresholds for TTFT and TPOT, respectively. Define

$$g_i = c_i \mathbf{1}\{D_i^{\text{first}} \leq \tau_f\} \mathbf{1}\{D_i^{\text{token}} \leq \tau_p\}, \quad G = \frac{1}{n} \sum_{i=1}^n g_i. \quad (1)$$

Attainment G measures the fraction of successful requests; SLO goodput $Q = \sum_i g_i / T$ measures their rate, where T runs from the first arrival until all requests complete or fail. Let $n_{\text{occ}}(t)$ count occupied replicas, including those starting or draining, and $\bar{C}$ denote their time-average count. We seek a feasible routing-and-scaling policy that maximizes J :

$$\bar{C} = \frac{1}{T} \int_0^T n_{\text{occ}}(t) dt, \quad J = \frac{Q}{R_{\text{ref}}} - \lambda_c \frac{\bar{C}}{C_{\text{max}}}, \quad (2)$$

where R_{ref} normalizes goodput, C_{max} is the maximum number of supported replicas, and λ_c controls the resource penalty. The objective balances timely service against replica occupancy, encouraging useful expansion while discouraging unnecessary resource use. Rewards track utility-potential increments, so the undiscounted return is proportional to J .

3. ADAPTIVE WORLD-MODEL PLANNING

WEF encodes the current serving state, evaluates candidate routing-and-scaling sequences, and executes one action. It screens candidates with short rollouts and selectively deepens promising trajectories with adaptive early stopping.

3.1. State encoding and candidate evaluation

Let o_i denote the observation for request i , comprising the request features and replica states described in Section 2. The encoder maps o_i to a latent representation $z_i = e_\theta(o_i)$. Dynamics f_θ predict the next latent state, r_θ predicts reward, and twin critics estimate the return beyond the rollout. A categorical policy prior π_θ guides action proposals.

At each planning iteration, WEF constructs N candidate action sequences with maximum horizon H . Of these, N_π follow the policy prior along predicted states; the remainder are sampled from a proposal initialized using the prior and the shifted previous plan. Action masks exclude infeasible routing and scaling decisions, such as routing to an unavailable replica or scaling during a cooldown period. For request i , candidate j starts from $z_0^j = e_\theta(o_i)$. The world model predicts successive latent states and accumulates rewards:

$$z_{k+1}^j = f_\theta(z_k^j, a_k^j), \quad R_j(h) = \sum_{k=0}^{h-1} \gamma^k r_\theta(z_k^j, a_k^j), \quad (3)$$

where k indexes rollout steps, a_k^j is candidate j 's action at step k , and $R_j(h)$ is its predicted cumulative reward through depth h , with discount factor γ . At depth h , the critics evaluate the prior's most probable action. Their mean $\bar{Q}_j(h)$ supplies the terminal value, and their population standard deviation $u_j(h)$ penalizes disagreement:

$$S_j(h) = R_j(h) + \gamma^h \bar{Q}_j(h) - \lambda_u u_j(h). \quad (4)$$

Thus $S_j(h)$ combines predicted rewards, terminal value, and an empirical uncertainty penalty, not a confidence bound.

3.2. Selective deepening and early exit

Not every candidate needs the same lookahead. WEF first evaluates all candidates to depth h_0 , then extends only the top F finalists. The retained candidates continue from depth h_0 without recomputing the preceding rollout steps. Other candidates retain their shallow scores and remain eligible for selection; comparisons across depths rely on the terminal-value estimates in (4).

Early stopping concerns the action executed now, not agreement over entire sequences. We therefore group candidates by their first action. With $\tilde{S}_j$ denoting the current score, define

$$V(a) = \max_{j: a_0^j = a} \tilde{S}_j, \quad m = V_{(1)} - V_{(2)}, \quad (5)$$

where $V_{(1)}$ and $V_{(2)}$ are the two highest distinct-action values; $m = 0$ if fewer than two actions are represented. Let j^* index the highest-scoring candidate. The gain $\Delta = \max(0, \tilde{S}_{j^*} - S_{\text{prev}})$ compares its score with the best score at the preceding check. After a finalist extension, deepening stops if

$$m \geq \tau_m, \quad u_{j^*} \leq \tau_u, \quad \Delta \leq \tau_g. \quad (6)$$

The thresholds τ_m , τ_u , and τ_g require a clear action margin, low critic disagreement, and little additional gain, respectively. Otherwise, rollout continues up to H . Requiring all three conditions prevents a large score margin alone from triggering early exit when critic disagreement remains high or further prediction still improves the best score.

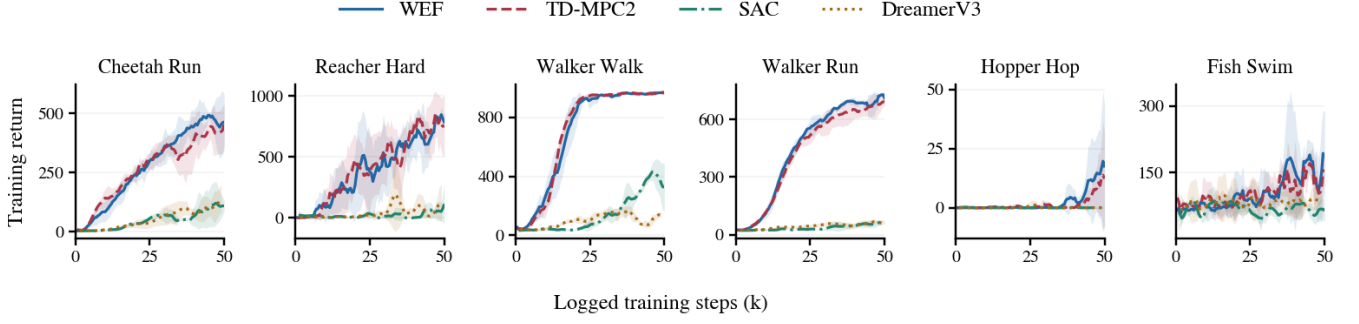


Fig. 1. Learning curves on six DMControl tasks over 50k logged training steps. Lines and shaded bands show the mean and ± 1 sample standard deviation of smoothed training returns across four seeds for WEF, TD-MPC2, SAC and DreamerV3.

Table 1. Episode returns on six DMControl tasks (mean $\pm$ standard deviation across seeds).

Task	WEF	TD-MPC2	SAC	DreamerV3
Cheetah Run	502.5 $\pm$ 84.8	462.9 $\pm$ 118.0	113.5 $\pm$ 29.3	138.0 $\pm$ 53.4
Reacher Hard	739.9 $\pm$ 174.8	733.9 $\pm$ 321.6	132.8 $\pm$ 186.6	113.7 $\pm$ 131.7
Walker Walk	971.4 $\pm$ 7.0	967.0 $\pm$ 8.3	332.0 $\pm$ 166.4	140.5 $\pm$ 20.8
Walker Run	706.7 $\pm$ 31.7	704.7 $\pm$ 57.3	65.5 $\pm$ 10.7	60.2 $\pm$ 13.8
Hopper Hop	12.8 $\pm$ 18.8	11.4 $\pm$ 15.9	0.0 $\pm$ 0.0	0.0 $\pm$ 0.1
Fish Swim	259.2 $\pm$ 157.8	197.5 $\pm$ 62.3	62.9 $\pm$ 24.6	83.1 $\pm$ 21.0

3.3. Action execution and planning cost

After scoring, the top K candidate sequences update the action-sampling distribution. This process repeats for up to I planning iterations. Unlike the F candidates retained for deeper prediction, these K elites guide sampling in the next iteration. Testing executes the best elite’s first action; training samples an elite according to its score and stores the resulting transition for replay. In one planning iteration, all N candidates advance to depth h_0 , and only F continue to the stopping depth $h \leq H$. The number of predicted state transitions is

$$M(h) = Nh_0 + F(h - h_0) + N_\pi H, \quad (7)$$

The three terms count shallow prediction, selective deepening, and prior-trajectory generation, respectively. Fixed-depth planning instead uses $NH + N_\pi H$ transitions per iteration. Screening reduces how many candidates continue, and early stopping reduces how far they advance. Total work sums these counts over the executed iterations.

4. EXPERIMENTS

We examine WEF on DMControl [22], evaluate its application to edge LLM serving and ablate its planning components.¹

4.1. Benchmark evaluation on DMControl

This experiment examines whether adaptive planning can support learning beyond the serving-specific setting. We compare the WEF with TD-MPC2, SAC [12], and DreamerV3 [23] on six DMControl

tasks. WEF uses Gaussian rather than categorical action proposals. Runs cover 50k logged steps; Figure 1 specifies the seed counts.

Figure 1 shows the training curves, while Table 1 compares the reported returns. WEF achieves the highest mean on all the six tasks. Its largest gains over TD-MPC2 occur on Cheetah Run and Fish Swim: returns increase about 8.6% and 31.2%, respectively. On Walker Walk and Walker Run, WEF preserves TD-MPC2’s strong performance with higher means and lower across-seed standard deviations. These results support the applicability of selective lookahead beyond serving-specific actions, without requiring every candidate to receive equally deep prediction. WEF also exceeds SAC and DreamerV3 in mean return on all six tasks.

4.2. Trace-driven serving setup

We evaluate joint routing and replica scaling in the eight-site system of Section 2. The simulator uses hardware-calibrated service curves for Qwen2.5-1.5B-Instruct. We set $C_{\max} = 16$, $\lambda_c = 0.1$, $\tau_f = 4$ s, $\tau_p = 0.12$ s, and $\gamma = 1$. Training/test timestamps are compressed to average 40/5 RPS for BurstGPT/WildChat, respectively, with concurrency capped at 8 per replica.

Workloads and protocol. BurstGPT provides 22,618 training requests and 11,696 test requests; WildChat provides 7,909 and 15,398, respectively. Each test set comprises three windows. Arrival-time rescaling preserves relative gaps and burst structure. WildChat supplies text-derived prefix identities and uses assistant-turn timestamps as arrival proxies; BurstGPT does not enable prefix reuse.

Each learning method trains online with four seeds. Frozen checkpoints are evaluated without parameter updates, and every window runs until accepted requests finish. We pool request counts, completed SLO successes, elapsed time, and container-seconds across windows before calculating G , Q , $\bar{C}$, and J for each seed.

Compared methods. PPO and SAC provide model-free references,

¹<https://github.com/ZhengYuhaoBUPT/WorldEdge-Flow-paper>

Table 2. Serving performance on BurstGPT and WildChat: SLO attainment G , goodput Q , mean replica occupancy $\bar{C}$, and utility J . Metrics pool three test windows per seed.

Method	BurstGPT				WildChat			
	G (%) $\uparrow$	Q (req/s) $\uparrow$	$\bar{C}$ $\downarrow$	J $\uparrow$	G (%) $\uparrow$	Q (req/s) $\uparrow$	$\bar{C}$ $\downarrow$	J $\uparrow$
WEF	88.93 $\pm$ 6.47	30.49 $\pm$ 3.69	10.15 $\pm$ 0.69	0.699 $\pm$ 0.095	80.45 $\pm$ 9.85	3.93 $\pm$ 0.47	9.03 $\pm$ 0.65	0.059 $\pm$ 0.017
TD-MPC2	87.52 $\pm$ 11.61	29.71 $\pm$ 5.89	11.77 $\pm$ 0.58	0.669 $\pm$ 0.144	75.34 $\pm$ 28.16	3.66 $\pm$ 1.41	8.70 $\pm$ 0.98	0.053 $\pm$ 0.041
PPO	58.52 $\pm$ 0.004	16.45 $\pm$ 0.001	9.00 $\pm$ 0.82	0.355 $\pm$ 0.005	26.24 $\pm$ 0.00	1.21 $\pm$ 0.00	8.00 $\pm$ 0.00	-0.015 $\pm$ 0.000
SAC	81.70 $\pm$ 10.85	26.86 $\pm$ 5.39	9.98 $\pm$ 0.80	0.609 $\pm$ 0.139	64.55 $\pm$ 24.58	3.13 $\pm$ 1.22	11.06 $\pm$ 1.47	0.023 $\pm$ 0.030
RR	79.87	24.56	8.00	0.564	75.37	3.65	8.00	0.058
HPA	79.72	24.76	13.45	0.535	72.85	3.56	8.94	0.053
JSQ	81.92	26.10	8.00	0.602	74.02	3.11	8.00	0.051

Table 3. Ablation of candidate screening and early stopping on BurstGPT using frozen checkpoints. Transitions/request counts candidate state transitions.

Planner	Attainment (%) $\uparrow$	Q (req/s) $\uparrow$	$\bar{C}$ $\downarrow$	J $\uparrow$	Transitions/request $\downarrow$
Fixed $H = 7$	84.80 $\pm$ 7.86	28.38 $\pm$ 3.49	10.71 $\pm$ 0.82	0.643 $\pm$ 0.084	6384.00 $\pm$ 0.00
Screening only	86.30 $\pm$ 9.28	29.65 $\pm$ 4.36	10.12 $\pm$ 0.47	0.678 $\pm$ 0.112	2664.00 $\pm$ 0.00
Early exit only	85.47 $\pm$ 6.89	28.48 $\pm$ 3.35	10.80 $\pm$ 0.97	0.644 $\pm$ 0.079	5279.38 $\pm$ 4.88
Full WEF	88.93 $\pm$ 6.47	30.49 $\pm$ 3.69	10.14 $\pm$ 0.69	0.699 $\pm$ 0.095	2462.01 $\pm$ 0.89

while TD-MPC2 uses fixed-horizon planning. WEF uses $H = 7$, $h_0 = 2$, $F = 4$, $(\tau_m, \tau_u, \tau_g) = (0.3, 0.03, 0.5)$, and $\lambda_u = 0.02$. Both planners use $(N, K, N_\pi) = (128, 16, 24)$, up to six refinement iterations, and training batches of 256. Round Robin (RR) cycles through eight resident replicas. Join the Shortest Queue (JSQ) uses the same fixed replicas and selects the smallest waiting-plus-active request count. Our queue-based Horizontal Pod Autoscaler (HPA) uses round-robin routing and activates the optional replica when the selected site’s waiting queue reaches 150% of one replica’s concurrency capacity; it requests scale-down below 50%.

4.3. SLO attainment and resource efficiency

Table 2 reports WEF’s highest mean attainment among the listed methods on both workloads. Mean attainment improves relative to TD-MPC2 by 1.61% on BurstGPT and 6.78% on WildChat. Mean utility rises from 0.669 to 0.699 and from 0.053 to 0.059, respectively.

Resource use distinguishes the two workloads. On BurstGPT, WEF improves attainment while reducing mean occupancy from TD-MPC2’s 11.77 containers to 10.15. On WildChat, its higher attainment uses slightly more capacity, 9.03 rather than 8.70 containers, but yields higher mean utility under the specified penalty. These results should be interpreted through both service quality and resource occupancy, rather than attainment alone.

The reported heuristics illustrate different resource tradeoffs. On BurstGPT, fixed-eight JSQ improves attainment over RR from 79.87% to 81.92%, whereas queue-based HPA uses 13.45 containers for 79.72% attainment. WEF reaches 88.93% with 10.15 containers. On WildChat, the listed RR, HPA, and JSQ attainment values are 75.37%, 72.85%, and 74.02%, respectively, compared with WEF’s 80.45%.

4.4. Frozen-planner ablation

To isolate candidate screening and early stopping, we change only the planner of each frozen WEF checkpoint and replay the three BurstGPT windows. Fixed $H = 7$ extends all candidates to depth seven. Screening only retains four candidates after depth two and extends them to seven. Early exit only applies the stopping rule without screening. Full WEF combines both mechanisms.

Table 4. Planning diagnostics under four-seed. Model steps count candidate transitions per request.

Metric	TD-MPC2	WEF
Plan time (ms) $\downarrow$	4.934 $\pm$ 0.000	1.485 $\pm$ 0.036
Decisions/s $\uparrow$	202.68 $\pm$ 0.00	673.84 $\pm$ 16.45
Ctrl. queue (ms) $\downarrow$	3.314 $\pm$ 0.000	0.997 $\pm$ 0.024
Model steps $\downarrow$	9648.00 $\pm$ 0.00	2462.01 $\pm$ 0.89
Selected depth	3.00 $\pm$ 0.00	5.67 $\pm$ 1.29

Table 3 reports relative mean attainment gains of 4.87%, 3.05%, and 4.05% for full WEF over fixed depth, screening only, and early exit only, respectively. Screening accounts for most of the computational reduction: transitions per request fall from 6,384 to 2,664, and adding early exit lowers the count further to 2,462. Thus, this workload supports a substantial reduction from screening and shows additional transition savings from early stopping.

Table 4 provides complementary evidence from the experiments. WEF reduces candidate transitions by 74.5%, from 9,648 to 2,462.01 per request, despite increasing the mean selected depth from 3.00 to 5.67. Selected depth describes the chosen trajectory, whereas transition count aggregates computation across candidates; selectively deepening a small subset can therefore yield longer lookahead with less total computation. The planning delay falls by 69.9%, with a corresponding 3.32 fold increase in modeled decision rate and lower controller queueing delay.

5. CONCLUSION

We presented WorldEdge-Flow, an adaptive world-model planner for joint routing and replica scaling. Candidate screening, prediction reuse, and early stopping concentrate lookahead on promising actions. DMControl benchmarks and hardware-calibrated BurstGPT and WildChat simulations show competitive control performance and higher mean SLO attainment and utility than the compared learning methods. Ablations attribute most transition savings to screening, supporting selective prediction for computation-efficient planning.

6. REFERENCES

- [1] Mahadev Satyanarayanan, “The emergence of edge computing,” *Computer*, vol. 50, no. 1, pp. 30–39, 2017.
- [2] Zhuohan Li, Lianmin Zheng, Yinmin Zhong, Vincent Liu, Ying Sheng, Xin Jin, Yanping Huang, Zhifeng Chen, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica, “AlpaServe: Statistical multiplexing with model parallelism for deep learning serving,” in *Proc. USENIX Symposium on Operating Systems Design and Implementation*, 2023, pp. 663–679.
- [3] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang, “DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving,” in *Proc. USENIX Symposium on Operating Systems Design and Implementation*, 2024.
- [4] Yuxin Wang, Yuhan Chen, Zeyu Li, Xueze Kang, Yuchu Fang, Yeju Zhou, Yang Zheng, Zhenheng Tang, Xin He, Rui Guo, Xin Wang, Qiang Wang, Amelie Chi Zhou, and Xiaowen Chu, “BurstGPT: A real-world workload dataset to optimize LLM serving systems,” *arXiv preprint arXiv:2401.17644v5*, 2025.
- [5] Wenting Zhao, Xiang Ren, Jack Hessel, Claire Cardie, Yejin Choi, and Yuntian Deng, “WildChat: 1m ChatGPT interaction logs in the wild,” in *Proc. International Conference on Learning Representations*, 2024.
- [6] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica, “Efficient memory management for large language model serving with PagedAttention,” in *Proc. ACM Symposium on Operating Systems Principles*, 2023, pp. 611–626.
- [7] Vikranth Srivatsa, Zijian He, Reyna Abhyankar, Dongming Li, and Yiyang Zhang, “Preble: Efficient distributed prompt scheduling for LLM serving,” *arXiv preprint arXiv:2407.00023*, 2024.
- [8] Yao Fu, Leyang Xue, Yeqi Huang, Andrei-Octavian Brabete, Dmitrii Ustiugov, Yuvraj Patel, and Luo Mai, “ServerlessLLM: Low-latency serverless inference for large language models,” in *Proc. USENIX Symposium on Operating Systems Design and Implementation*, 2024, pp. 135–153.
- [9] Hongzi Mao, Mohammad Alizadeh, Ishai Menache, and Srikanth Kandula, “Resource management with deep reinforcement learning,” in *Proc. ACM Workshop on Hot Topics in Networks*, 2016.
- [10] Hongzi Mao, Malte Schwarzkopf, Shaileshh Bojja Venkatakrishnan, Zili Meng, and Mohammad Alizadeh, “Learning scheduling algorithms for data processing clusters,” in *Proc. ACM SIGCOMM*, 2019, pp. 270–288.
- [11] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.
- [12] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine, “Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor,” in *Proc. International Conference on Machine Learning*, 2018, pp. 1861–1870.
- [13] Kurtland Chua, Roberto Calandra, Rowan McAllister, and Sergey Levine, “Deep reinforcement learning in a handful of trials using probabilistic dynamics models,” in *Advances in Neural Information Processing Systems*, 2018, vol. 31.
- [14] Danijar Hafner, Timothy Lillicrap, Ian Fischer, Ruben Villegas, David Ha, Honglak Lee, and James Davidson, “Learning latent dynamics for planning from pixels,” in *Proc. International Conference on Machine Learning*, 2019, pp. 2555–2565.
- [15] Nicklas A. Hansen, Hao Su, and Xiaolong Wang, “Temporal difference learning for model predictive control,” in *Proc. International Conference on Machine Learning*, 2022, pp. 8387–8406.
- [16] Nicklas Hansen, Hao Su, and Xiaolong Wang, “TD-MPC2: Scalable, robust world models for continuous control,” in *Proc. International Conference on Learning Representations*, 2024.
- [17] Arthur J. Krener, “Adaptive horizon model predictive control,” *arXiv preprint arXiv:1602.08619*, 2016.
- [18] Michael Janner, Justin Fu, Marvin Zhang, and Sergey Levine, “When to trust your model: Model-based policy optimization,” in *Advances in Neural Information Processing Systems*, 2019, vol. 32.
- [19] Surat Teerapittayanon, Bradley McDanel, and H. T. Kung, “BranchyNet: Fast inference via early exiting from deep neural networks,” in *Proc. International Conference on Pattern Recognition*, 2016, pp. 2464–2469.
- [20] Stefanos Laskaridis, Alexandros Kouris, and Nicholas D. Lane, “Adaptive inference through early-exit networks: Design, challenges and directions,” in *Proc. International Workshop on Embedded and Mobile Deep Learning*, 2021.
- [21] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, et al., “SGLang: Efficient execution of structured language model programs,” *arXiv preprint arXiv:2312.07104*, 2023.
- [22] Yuval Tassa, Yotam Doron, Alistair Muldal, Tom Erez, Yazhe Li, Diego de Las Casas, David Budden, Abbas Abdolmaleki, Josh Merel, Andrew Lefrancq, Timothy Lillicrap, and Martin Riedmiller, “DeepMind control suite,” *arXiv preprint arXiv:1801.00690*, 2018.
- [23] Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap, “Mastering diverse control tasks through world models,” *Nature*, vol. 640, pp. 647–653, 2025.